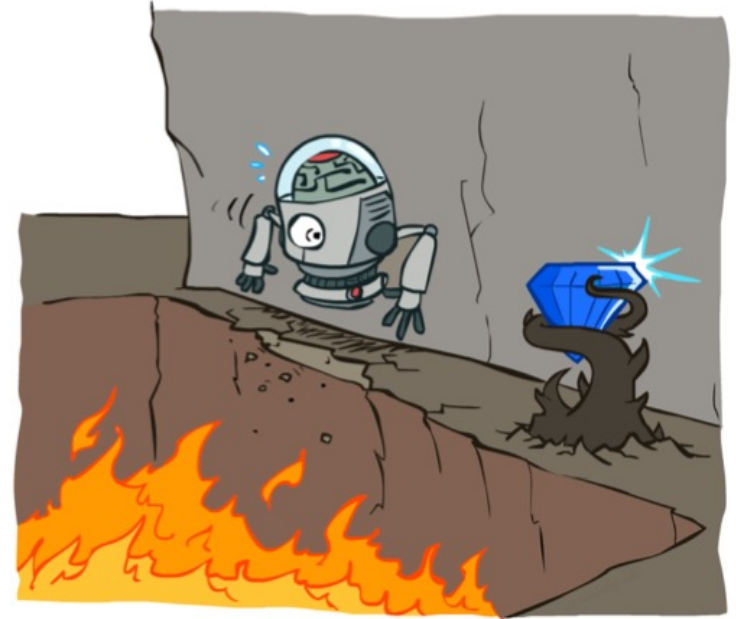


Artificial Intelligence Laboratory 4: Reinforcement Learning

DT8042 Halmstad University
Dec 2021

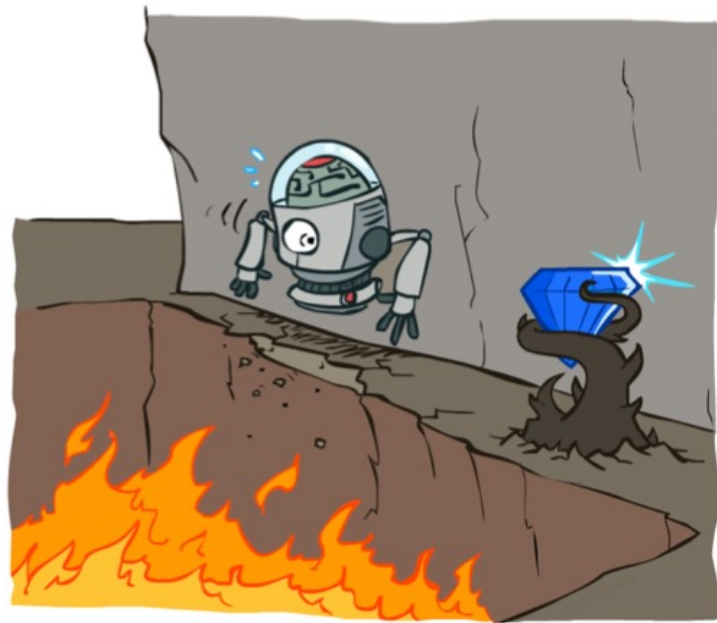
Lab 4

- Non-deterministic search in stochastic grid world
- Value and Policy Iteration
- Temporal Difference Learning and Q-Learning



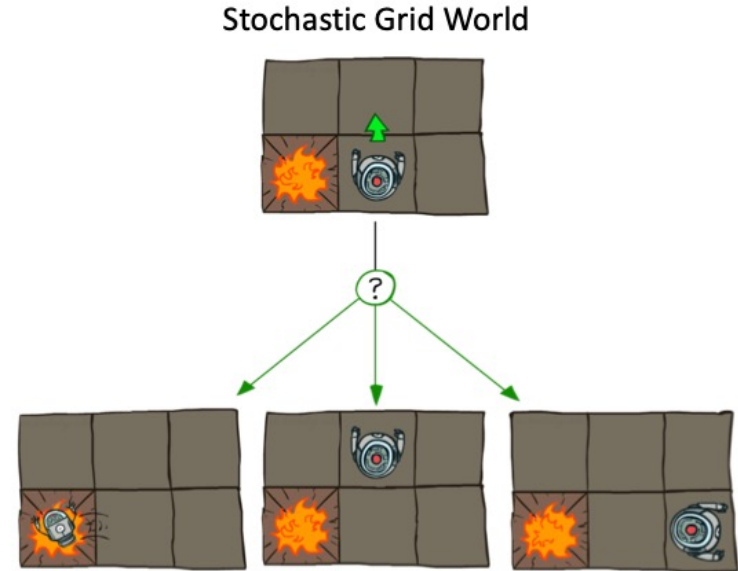
Overview

- Review core concepts
 - Stochastic grid world
 - Markov Decision Process
 - Policies, Utility and discounting
 - The Bellman Equation
- Tasks
 - Value and policy iteration
 - Q-learning and TD-Learning
- Implementation details
- Experiments
- Grading criteria



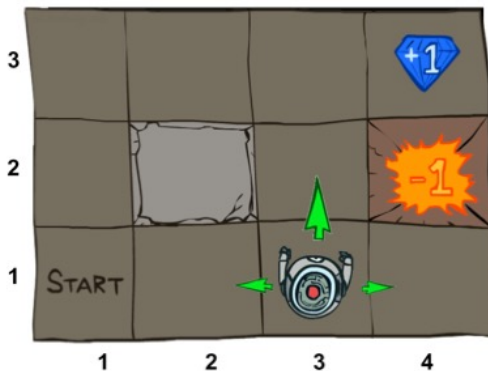
Environment: Grid World

- Representing the grid with matrix
- Non-deterministic state transition
 - 80% perform action as intended
 - 10% towards each side directions
 - Stay put if the cell is not accessible
- Rewards were received over time
 - Small living rewards
 - Big reward reaching the terminal state



Markov Decision Process

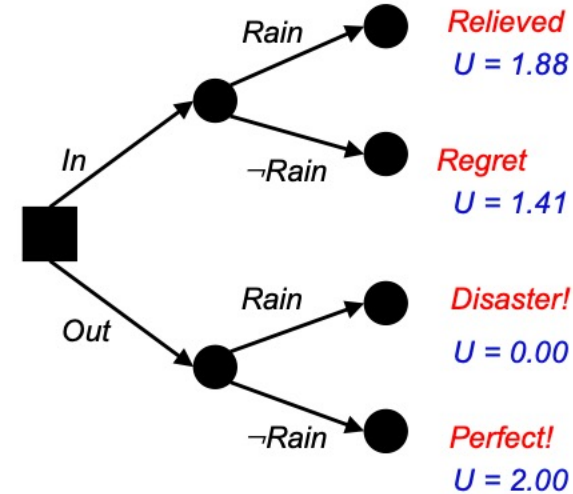
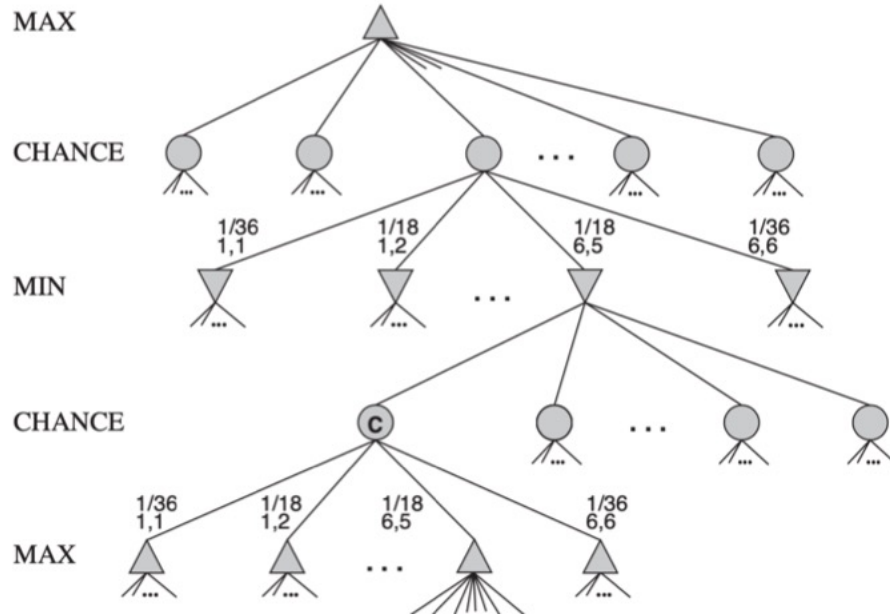
- An MDP is defined by
 - A set of states
 - A set of actions
 - A (state) transition function $T(s, a, s')$
 - Probability that perform action a at s leads to s'
 - Model/dynamics
 - A reward function $R(s, a, s')$
 - Or just $R(s)$ or $R(s')$
 - A starting state
 - Maybe a terminal state



$$\begin{aligned} &P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \dots, S_0 = s_0) \\ &= \\ &P(S_{t+1} = s' | S_t = s_t, A_t = a_t) \end{aligned}$$

- Expectimax search

Expectimax search



Policies, Utility and discounting

- Deterministic search
 - optimal sequence of actions
- For non-deterministic search, e.g. MDP
 - Policy $\pi^*: S \rightarrow A$
 - A policy gives an action for each state
 - An optimal policy maximizes the utility
- Utility of sequence
 - Discounting factor

Policies, Utility and discounting

- optimal quantities
 - $V^*(s)$ the expected utility of a state s , acting optimally
 - $Q^*(s, a)$ the expected utility of a q state (s, a) , taking the optimal action a
 - $\pi^*(s)$ optimal action from state s
- The bellman equations

$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

Tasks

- Get familiar with the environment and model
- Read the code of iteration and Q-learning
- Create two more grids of moderate size for analysis
- Implement policy iteration and TD-learning
- Performs a set of experiments and answers the questions
 - Compare convergence time of value and policy iteration
 - Study the impact of immediate reward to optimal policy
 - Compare the results
 - Value vs. policy iteration
 - TD-learning vs. Q-learning
 - Study the impact of epsilon, alpha values, and numbers of episode chosen

Value Iteration

- For each state s , compute the expected reward of being in s and acts optimally
 - Compute the utility value using the Bellman equation

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- The algorithm is provided in the note book
 - Initialize $V^*(s)$
 - While not converged
 - Compute $V^*(s)$ for each state s

Policy Iteration

- Why policy iteration?
 - May converge faster, compared to value iteration
- Implement the algorithm
 - Initialization $V(s)$ and $\pi(s)$ for all state s
 - Policy evaluation
 - Compute values with fixed current policy $\pi(s)$, iterate until converge

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$

- Policy improvement
 - Update with a better policy using policy extraction

$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$

Policy Iteration

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Repeat

$\Delta \leftarrow 0$

For each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s'} \mathcal{P}_{ss'}^{\pi(s)} [\mathcal{R}_{ss'}^{\pi(s)} + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

$b \leftarrow \pi(s)$

$\pi(s) \leftarrow \arg \max_a \sum_{s'} \mathcal{P}_{ss'}^a [\mathcal{R}_{ss'}^a + \gamma V(s')]$

If $b \neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop; else go to 2

- Policy evaluation

- Compute values with fixed current policy $\pi_i(s)$, iterate until converge

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$

- Policy improvement

- Update with a better policy using policy extraction

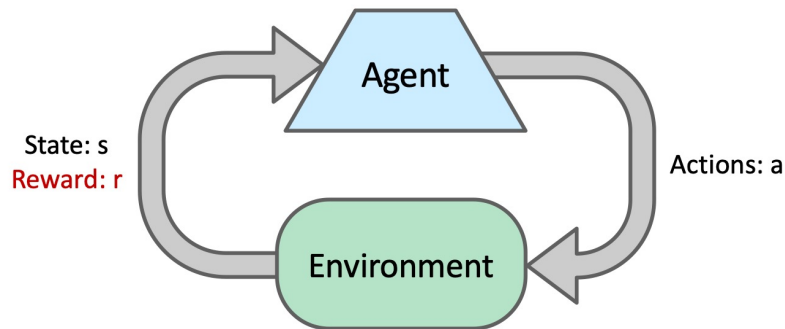
$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$

Q-Learning

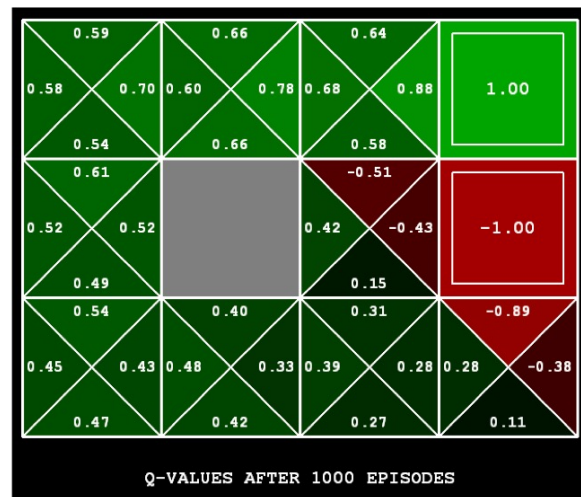
- What if the model is unknown
 - T or R need to be learned
 - Off-line planning vs. online ~~planning~~ learning
 - Exploration & Exploitation
 - Regret
 - Sampling
 - Much harder than solving MDP

- Sample-based Q-value iteration

- Receive a sample (s, a, s', r)
- New sample estimation $sample = R(s, a, s') + \gamma \max_{a'} Q(s', a')$
- Incorporate the estimate $Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + (\alpha) [sample]$



$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma \max_{a'} Q_k(s', a') \right]$$



Temporal Difference Learning

- Objective
 - Predict the expected total reward over future time
- Learn from experience
 - Sampling new states from the environment and update $V(s)$ for new transition (s, a, s', r)

$$V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + (\alpha)\textit{sample}$$

$$V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha [R(s, \pi(s), s') + \gamma V^\pi(s')]$$

- Monte-Carlo sampling

Temporal Difference Learning

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated

Algorithm parameter: step size $\alpha \in (0, 1]$

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

$A \leftarrow$ action given by π for S

 Take action A , observe R, S'

$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

 until S is terminal

Tasks

- Get familiar with the environment and model
- Read the code of iteration and Q-learning
- Create two more grids of moderate size for analysis
- Implement policy iteration and TD-learning
- Performs a set of experiments and answers the questions
 - Compare convergence time of value and policy iteration
 - Study the impact of immediate reward to optimal policy
 - Compare the results
 - Value vs. policy iteration
 - TD-learning vs. Q-learning
 - Study the impact of epsilon, alpha values, and numbers of episode chosen

Grading

- Submit your assignment on the Blackboard
 - A short report about what you have done
 - Answer to the questions
 - Task 1-3 for completing the lab
 - Task 4 for extra credits
- Deadline 2022-Jan-7